



Facultad de Ciencias Exactas, Físicas y Naturales
Universidad Nacional de San Juan

fcefn



DEPARTAMENTO
DE INFORMÁTICA

Trabajo Práctico 4: Teoría de la información

**CODIFICACION – HUFFMAN, SHANNON,
FANO**

Profesores:

Mag. Ing. Raúl Oscar Klenzi

Lic. Manuel Oscar Ortega

Lic. Fabrizio Amaya

Alumnos:

Figueroa Santiago E010-186

Ríos Matias E010-133

Zalazar Lucas E010-340

EJERCICIO 1. Comprobar que $H_r \leq LH$

Para comprobarlo, calculamos la entropía y luego la longitud media de un código binario óptimo obtenido mediante Huffman.

a) Probabilidades: $p = \{0.4, 0.2, 0.2, 0.1, 0.1\}$

Entropía

$$H = - [0.4\log_2(0.4) + 0.2\log_2(0.2) + 0.2\log_2(0.2) + 0.1\log_2(0.1) + 0.1\log_2(0.1)]$$

$$H = 2.122 \text{ bits / símbolo}$$

Huffman

Huffman produce longitudes posibles: $l = \{1, 2, 3, 3, 4\}$

asignadas a las probabilidades correspondientes de forma que las más probables tengan códigos más cortos.

Entonces:

$$L = 0.4(1) + 0.2(2) + 0.2(3) + 0.1(4) + 0.1(4) = 2.2$$

Por lo tanto:

$$2.122 \leq 2.2 \rightarrow \text{Se cumple}$$

b)

$$p = \{0.4, 0.4, 0.1, 0.1\}$$

Entropía

$$H = - [2(0.4\log_2 0.4) + 2(0.1\log_2 0.1)]$$

$$H = 1.722 \text{ bits / símbolo}$$

Huffman

$$L = \{1, 2, 3, 3\}$$

Entonces:

$$L = 0.4(1) + 0.4(2) + 0.1(3) + 0.1(3) = 1.8$$

Por lo tanto:

$$1.722 \leq 1.8 \rightarrow \text{Se cumple}$$

c)

$$p = \{0.4, 0.3, 0.2, 0.1\}$$

Entropía

$$H = - [0.4\log_2(0.4) + 0.3\log_2(0.3) + 0.2\log_2(0.2) + 0.1\log_2(0.1)]$$

$$H = 1.846 \text{ bits / símbolo}$$

Huffman:

$$L = \{1, 2, 3, 3\}$$

$$L = 0.4(1) + 0.3(2) + 0.2(3) + 0.1(3) = 1.9$$

Entonces:

$$1.846 \leq 1.9 \rightarrow \text{Se cumple}$$

EJERCICIO 2. Fuente con 4 eventos equiprobables

$$\text{Tenemos: } p(s_1) = p(s_2) = p(s_3) = p(s_4) = 1/4$$

La entropía es:

$$H = - 4(1/4\log_2 1/4) = 2 \text{ bits / símbolo}$$

$$\text{Como: } H(S) \leq L$$

$$\text{entonces: } 2 \leq L$$

Por lo tanto: $L = 1.86$ no es posible

$$\text{El mínimo es: } L_{\min} = 2$$

Por ejemplo:

$$s_1 = 00, s_2 = 01, s_3 = 10, s_4 = 11$$

EJERCICIO 3. Fuente con $p(s_1) = 1/2$, $p(s_2) = 1/4$, $p(s_3) = p(s_4) = 1/8$

$$\text{Tenemos: } p = \{1/2, 1/4, 1/8, 1/8\}$$

Entropía

$$H = - [1/2\log_2 1/2 + 1/4\log_2 1/4 + 2(1/8\log_2 1/8)]$$

$$H = 1.75 \text{ bits / símbolo}$$

Huffman

Combinamos los dos menores:

$$1/8 + 1/8 = 1/4$$

Luego:

$$1/4 + 1/4 = 1/2$$

Código posible:

Símbolo	Probabilidad	Código	Longitud
s ₁	1/2	0	1
s ₂	1/4	10	2
s ₃	1/8	110	3
s ₄	1/8	111	3

La longitud media:

$$L_H = 1/2(1) + 1/4(2) + 1/8(3) + 1/8(3) = 1.75$$

Por lo tanto:

$$H = L = 1.75 \text{ bits / símbolo}$$

En este caso el código alcanza exactamente la entropía.

¿Es posible $L=1,86$?

No.

Como las longitudes son enteras:

$$L = 1/2l_1 + 1/4l_2 + 1/8l_3 + 1/8l_4$$

$$L = (4l_1 + 2l_2 + l_3 + l_4) / 8$$

por lo que los valores posibles están separados por múltiplos de $1/8 = 0,125$

Por ejemplo:

$$1,75, 1,875, 2, \dots$$

pero: 1,86 no aparece.

Por lo tanto: $L = 1,86$ no es posible

y el mínimo es: $L_{\min} = 1,75$

EJERCICIO 4. Código trinario mediante Huffman

Tenemos: $S = (s_1, s_2, s_3, s_4, s_5, s_6, s_7)$

con:

$$p_1 = p_2 = 1/3; p_3 = p_4 = 1/9; p_5 = p_6 = p_7 = 1/27$$

Como el código es ternario, la base es:

$$r = 3$$

Huffman ternario

Primero:

$$1/27 + 1/27 + 1/27 = 1/9$$

Ahora tenemos:

$$1/3, 1/3, 1/9, 1/9, 1/9$$

Combinamos los tres $1/9$:

$$1/9 + 1/9 + 1/9 = 1/3$$

Finalmente:

$$1/3 + 1/3 + 1/3 = 1$$

Una codificación posible:

Símbolo	Probabilidad	Código ternario	Longitud
s1	1/3	0	1
s2	1/3	1	1
s3	1/9	20	2
s4	1/9	21	2
s5	1/27	220	3
s6	1/27	221	3
s7	1/27	222	3

Entropía

En bits:

$$H(S) = 2,2894 \text{ bits/símbolo}$$

Pero como el código es ternario, conviene expresarla en trits:

$$H_3(S) = H_2(S) / \log_2 3$$

$$H_3(S) = 1,4444 \text{ trits / símbolo}$$

Longitud media

$$L = 2(1/3)(1) + 2(1/9)(2) + 3(1/27)(3)$$

$$L = 2/3 + 4/9 + 1/3$$

$$L = 1,4444 \text{ trits/símbolo}$$

Entonces:

$$H_3(S) = L = 1,4444$$

EJERCICIO 5. Huffman, Shannon y Fano

Fuente:

$$p(s_1) = 1/2, p(s_2) = 1/4, p(s_3) = 1/4$$

Huffman

$$s_1 = 0, s_2 = 10, s_3 = 11$$

Entonces:

$$L_H = 1/2(1) + 1/4(2) + 1/4(2) = 1.5$$

Shannon

Las longitudes son: $l_i = \lceil \log_2 p_i \rceil$

Por lo tanto:

$$l_1 = 1; l_2 = l_3 = 2$$

Código posible:

$$s_1 = 0, s_2 = 10, s_3 = 11$$

Entonces:

$$L_S = 1.5$$

Fano

Ordenamos de mayor a menor:

$1/2, 1/4, 1/4$

Dividimos en grupos:

$\{s_1\} \mid \{s_2, s_3\}$

Código posible:

$s_1 = 0, s_2 = 10, s_3 = 11$

Entonces:

$L_F = 1.5$

Resultado final:

$L_H = L_S = L_F = 1.5$ bits

Como las probabilidades son potencias exactas de $1/2$, los tres métodos alcanzan la entropía máxima ideal.

EJERCICIO 6. Segunda y tercera extensión

La fuente original es:

$p_1 = 1/2; p_2 = p_3 = 1/4$

y:

$H(S) = 1,5$

Segunda extensión

Hay: $3^2 = 9$ combinaciones.

Las probabilidades son:

Probabilidad	Cantidad
$1/4$	1
$1/8$	4
$1/16$	4

Aplicando Huffman se obtienen longitudes posibles:

Probabilidad	Cantidad	Longitud
$1/4$	1	2
$1/8$	4	3
$1/16$	4	4

Entonces:

$$L_2 = 1/4(2) + 4(1/8)(3) + 4(1/16)(4)$$

$$L_2 = 3 \text{ bits / bloque}$$

Como el bloque tiene 2 símbolos:

$$L_2/2 = 3/2 = 1.5 \text{ bits / símbolo}$$

Tercera extensión

Ahora tenemos: $3^3 = 27$ combinaciones.

Las probabilidades son:

Probabilidad	Cantidad
1/8	1
1/16	6
1/32	12
1/64	8

Huffman produce longitudes:

Probabilidad	Cantidad	Longitud
1/8	1	3
1/16	6	4
1/32	12	5
1/64	8	6

Entonces:

$$L_3 = 1/8(3) + 6(1/16)(4) + 12(1/32)(5) + 8(1/64)(6)$$

$$L_3 = 4.5 \text{ bits/bloque}$$

Como cada bloque tiene 3 símbolos:

$$4.5/3 = 1.5$$

Tendencia

Fuente	Bits por bloque	Bits por símbolo
Original	1.5	1.5
2.ª extensión	3	1.5

3.ª extensión	4.5	1.5
----------------------	-----	-----

Se observa que la longitud total aumenta, porque cada bloque contiene más símbolos, pero la longitud promedio por símbolo permanece en 1.5 bits.

Esto muestra la tendencia de la longitud de la fuente original.

EJERCICIO 7. Analisis de cadena de texto

Texto: **PABLITOPABLITOPABLOCLAVO**

El texto tiene: 24 símbolos

y las frecuencias son:

Símbolo	Frecuencia
A	4
L	4
O	4
B	3
P	3
I	2
T	2
C	1
V	1

Como hay 9 símbolos distintos, un código binario de longitud fija necesita:

$$\log_2 9 = 4 \text{ bits / símbolo}$$

Por lo tanto, sin compresión:

$$24(4) = 96 \text{ bits}$$

a) Codificar la segunda extensión con Fano

Tomamos pares consecutivos:

PA, BL, IT, OP, AB, LI, TO, PA, BL, OC, LA, VO

Tenemos 12 pares.

Frecuencias:

Par	Frecuencia
PA	2
BL	2

IT	1
OP	1
AB	1
LI	1
TO	1
OC	1
LA	1
VO	1

Ordenamos de mayor a menor, como indica el método de Fano. La teoría nos indica precisamente que se ordenan las probabilidades y luego se dividen en grupos tratando de que sean equiprobables.

Una codificación Fano válida es:

Par	Código
PA	00
BL	010
IT	0110
OP	0111
AB	100
LI	1010
TO	1011
OC	110
LA	1110
VO	1111

Cantidad de bits:

$$2(2) + 2(3) + 6(4) = 40$$

y la longitud promedio por par:

$$L_F = 40/12 = 3.333 \text{ bits/par}$$

Como cada para representa dos símbolos:

$$L_F = 1.667 \text{ bits/símbolo}$$

b) Porcentaje de compresión

Original: 96 bits

Comprimido: 40 bits

Entonces:

$$\text{Compresión} = ((96 - 40) / 96) \times 100$$

$$\text{Compresión} = 58,33\%$$

Ahora como fuente de Markov de orden 1

Las transiciones de nuestro texto son:

Estado actual	Próximos símbolos
P	A,A,A
A	B,B,B,V
B	L,L,L
L	I,I,O,A
I	T,T
T	O,O
O	P,P,C
C	L
V	O

Códigos Fano para cada estado

Desde P

Solamente: $P \rightarrow A$

Código: $A = 0$

Desde A

$A \rightarrow B : 3/4$

$A \rightarrow V : 1/4$

Dos símbolos $\rightarrow$ 1 bit cada uno:

$B = 0, V = 1$

Desde B

Solamente:

$B \rightarrow L$

$L = 0$

Desde L

$L \rightarrow I : 2/4$

$L \rightarrow O : 1/4$

$L \rightarrow A : 1/4$

Fano:

$I = 0, O = 10, A = 11$

Desde I

$I \rightarrow T$

$T = 0$

Desde T

$T \rightarrow O$

$O = 0$

Desde O

$O \rightarrow P : 2/3$

$O \rightarrow C : 1/3$

$P = 0, C = 1$

Desde C

$C \rightarrow L$

$L = 0$

Desde V

$V \rightarrow O$

$O = 0$

Codificación del texto

El primer símbolo es:

P

Como hay 9 símbolos posibles, usamos 4 bits:

$P \rightarrow 0000$

Después codificamos cada transición según el estado anterior.

La cantidad de bits de las 23 transiciones es:

- $P \rightarrow A : 1$
- $A \rightarrow B, B, B, V : 4$
- $B \rightarrow L, L, L : 3$
- $L \rightarrow I, O, A, I : 1 + 2 + 2 + 1 = 6$
- $I \rightarrow T, TI : 2$
- $T \rightarrow O, OT : 2$
- $O \rightarrow P, P, C : 3$
- $C \rightarrow L : 1$
- $V \rightarrow O : 1$

Total:

$$1 + 4 + 3 + 6 + 2 + 2 + 3 + 1 + 1 = 23$$

Sumando los 4 bits del primer símbolo:

$$L_{\text{Markov}} = 27 \text{ bits}$$

Compresión

$$((96 - 27)/96) \times 100$$

$$\text{Compresión Markov} = 71,875\%$$

Comparación

Método	Bits utilizados	Compresión
Fano 2.ª extensión	40 bits	58,33 %
Fano + Markov orden 1	27 bits	71,88 %

c) ¿Por qué existe tanta diferencia?

La diferencia se debe a que el método de Markov **no considera los símbolos como independientes**.

En el texto aparecen muchas transiciones muy repetidas, por ejemplo:

P → A

B → L

I → T

T → O

y otras tienen muy pocas posibilidades.

En una fuente de Markov el código del símbolo siguiente depende del **estado anterior**.

Por eso, el modelo de Markov aprovecha información que el Fano de la segunda extensión no aprovecha de la misma manera.

EJERCICIO 8. Pipeline BWT → RLE → Huffman

Cadena: **TOMATOMATOMA\$**

BWT:

1. genera las rotaciones;
2. las ordena lexicográficamente;
3. toma la última columna.

Además, \$ debe considerarse como carácter especial de fin de cadena y menor que los demás caracteres.

a) RLE directamente

La cadena es:

T O M A T O M A T O M A \$

No hay caracteres repetidos consecutivamente.

Entonces RLE produce:

T1 O1 M1 A1 T1 O1 M1 A1 T1 O1 M1 A1 \$1

Tenemos:

13 corridas

Por lo tanto, **RLE directo prácticamente no consigue compresión.**

b) BWT

Cadena:

TOMATOMATOMA\$

Las rotaciones ordenadas son:

1. \$TOMATOMATOMA
2. A\$TOMATOMATOM
3. ATOMA\$TOMATOM
4. ATOMATOMA\$TOM
5. MA\$TOMATOMATO
6. MATOMA\$TOMATO
7. MATOMATOMA\$TO
8. OMA\$TOMATOMAT
9. OMATOMA\$TOMAT
10. OMATOMATOMA\$T
11. TOMA\$TOMATOMA
12. TOMATOMA\$TOMA
13. TOMATOMATOMA\$

Tomamos la última columna:

A M M M O O O T T T A A \$

Por lo tanto:

BWT = AMMMOOOTTTAA\$

Esto es justamente lo que busca BWT: reorganizar los caracteres para crear corridas.

Aplicamos RLE

La salida:

A M M M O O O T T T A A \$

se transforma en:

A1 M3 O3 T3 A2 \$1

Ahora tenemos solamente:

6 corridas

Comparación

Método	Resultado RLE	Corridas
RLE directo	T1 O1 M1 A1 T1 O1 M1 A1 T1 O1 M1 A1 \$1	13
BWT + RLE	A1 M3 O3 T3 A2 \$1	6

La reducción de corridas es:

$$((13 - 6) / 13) \times 100 = 53,85\%$$

Es decir, BWT consiguió reducir la cantidad de corridas de **13 a 6**.

BWT no comprime por sí misma; reorganiza la información para que RLE y/o Huffman puedan comprimirla mejor.